

D: > Career tracks > Digial_IC > 09-Verification > Lab1_1 > Adder > V adder.v

```
1  module adder (
2      input  clk,
3      input  reset,
4      input  signed [3:0] A, // Input data A in 2's complement
5      input  signed [3:0] B, // Input data B in 2's complement
6      output reg signed [4:0] C // Adder output in 2's complement
7  );
8
9      // Register output C
10     always @(posedge clk or posedge reset) begin
11         if (reset)
12             C <= 5'b0;
13         else
14             C <= A + B;
15     end
16
17 endmodule
18
```

1

```
module adder_tb;

    // Declare the stimulus signals and the response signal
    reg clk_tb;
    reg reset_tb;
    reg signed [3:0] A;
    reg signed [3:0] B;
    wire signed [4:0] C;
    // Declare error_count and correct_count as integer.
    int error_count, correct_count;

    // Declare localparams MAXPOS and MAXNEG
    localparam MAXPOS = 7,
               MAXNEG = -8;
    // Instantiate the design under test (adder).
    adder uut(
        clk_tb,
        reset_tb,
        A,
        B,
        C
    );

    // Write a clock generator block that toggles clk_tb every 5 time units.
    always begin
        #5 clk_tb = ~clk_tb;
    end

    task check_result(int expected_sum);
        @(negedge clk_tb);
        if (expected_sum != C) begin
            error_count += 1;
            $display("A = %d, B = %d, expected = %d, Actual = %d",A,B,expected_sum,C);
        end
        else
            correct_count += 1;
    endtask
endtask
```

```

task assert_reset;
    reset_tb = 1;
    check_result(0);
    reset_tb = 0;
endtask

initial begin
    clk_tb = 0;
    A = 0;
    B = 0;
    reset_tb = 0;
    correct_count = 0;
    error_count = 0;

    #5;
    // call assert_reset
    assert_reset;

    #5;
    // call assert_reset
    assert_reset;

    A = 0;
    B = 0;
    check_result(0);
    B = MAXPOS;
    check_result(MAXPOS);
    B = MAXNEG;
    check_result(MAXNEG);

    A = MAXPOS;
    B = 0;
    check_result(MAXPOS);
    B = MAXPOS;
    check_result(MAXPOS + MAXPOS);
    B = MAXNEG;
    check_result(MAXPOS + MAXNEG);

```

```

    A = MAXNEG;
    B = 0;
    check_result(MAXNEG);
    B = MAXPOS;
    check_result(MAXPOS + MAXNEG);
    B = MAXNEG;
    check_result(MAXNEG);

    A = 0;
    B = 0;
    check_result(0);

    A = 4'b1001;
    B = 4'b1010;
    check_result(5'b10011);
    $display("Error count = %d and Correct count = %d",error_count,correct_count);
    $stop;
end

endmodule

```

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ENCODER_1	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ENCODER_2	Applying all possible valid values of D	Directed during the simulation	-	A checker in the testbench to make sure the output is correct

```

1  module priority_enc (
2  input  clk,
3  input  rst,
4  input  [3:0] D,
5  output reg [1:0] Y,
6  output reg valid
7  );
8
9  always @(posedge clk) begin
10     if (rst) begin
11         Y <= 2'b0;
12         valid <= 1'b0;
13     end
14     else
15         casex (D)
16             4'b1000: Y <= 0;
17             4'bX100: Y <= 1;
18             4'bXX10: Y <= 2;
19             4'bXXX1: Y <= 3;
20         endcase
21         valid <= (~|D)? 1'b0: 1'b1;
22     end
23 endmodule

```

```
module priority_enc_tb;          You, last month • verification tutorial
```

```
    // Declare local wire and reg
    reg clk_tb;
    reg reset_tb;
    reg [3:0] D_tb;
    wire [1:0] Y_tb;
    wire valid_tb;
```

```
    // Instantiate the module under test
```

```
    priority_enc uut(
        clk_tb,
        reset_tb,
        D_tb,
        Y_tb,
        valid_tb
    );
```

```
    // define local parameter for tracking the functionality
```

```
    int count_correct, count_error;
```

```
    // define clock
```

```
    localparam T = 20;
```

```
    always begin
```

```
        clk_tb = 1'b0;
```

```
        #(T/2);
```

```
        clk_tb = 1'b1;
```

```
        #(T/2);
```

```
    end
```

```
    // define task for checking
```

```
    task checking(int expected_result, expected_validation);
```

```
        @(negedge clk_tb);
```

```
        if (expected_result != Y_tb | expected_validation != valid_tb) begin
```

```
            count_error += 1;
```

```
            $display("A = %d, expected_out = %d, Actual_out = %d, expected_valid = %d, Actual_valid = %d", D_tb, expected_result, Y_tb, expected_validation, valid_tb);
```

```
        end
```

```
    else begin
```

```
        count_correct += 1;
```

```
    end
```

```
endtask
```

```
    // define task for reset functionality
```

```
    task assert_reset;
```

```
        reset_tb = 1'b1;
```

```
        checking(2'b00, 1'b0);
```

```
        reset_tb = 1'b0;
```

```
    endtask
```

```
    initial begin
```

```
        clk_tb = 1'b0;
```

```
        reset_tb = 1'b0;
```

```
        D_tb = 4'b0;
```

```
        checking(2'bxx, 1'b0);
```

```
        #10;
```

```
        assert_reset;
```

```
        #10;
```

```
        assert_reset;
```

```
        #10;
```

```
        D_tb = 4'b0001;
```

```
        checking(2'b11, 1'b1);
```

```
        #10;
```

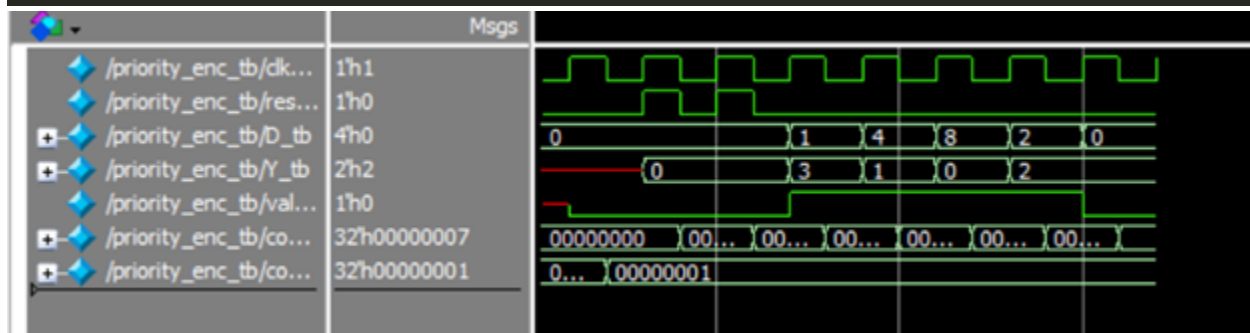
```
        D_tb = 4'b0100;
```

```
        checking(2'b01, 1'b1);
```

```

70     #10;
71     D_tb = 4'b1000;
72     checking(2'b00,1'b1);
73
74     #10;
75     D_tb = 4'b0010;
76     checking(2'b10,1'b1);
77
78     #10;
79     D_tb = 4'b0000;
80     checking(2'b10,1'b0);
81
82     #10;
83     $$display("Correct count = %d and Error count = %d",count_correct,count_error);
84     $stop;
85 end
86
87 endmodule

```



```

=== Instance: /priority_enc_tb/uut
=== Design Unit: work.priority_enc

```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Branches	7	7	0	100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	7	7	0	100.00%

=====Statement Details=====

=====Toggle Details=====

Toggle Coverage for instance /priority_enc_tb/uut --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
D[0-3]	1	1	100.00
Y[1-0]	1	1	100.00
<u>clk</u>	1	1	100.00
<u>rst</u>	1	1	100.00

Total Node Count = 8

Toggled Node Count = 8

Untoggled Node Count = 0

Toggle Coverage = 100.00% (16 of 16 bins)

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALU_1	When the reset is asserted, the C value should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ALU_2	Verifying maximum negative value of A and maximum negative value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_3	Verifying maximum negative value of A and maximum negative value of B while opcode is SUB	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_4	Verifying maximum negative value of A and maximum zero value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_5	Verifying maximum negative value of A and maximum zero value of B while opcode is SUB	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_6	Verifying maximum negative value of A and maximum positive value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_7	Verifying maximum negative value of A and maximum positive value of B while opcode is SUB	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_8	Verifying maximum positive value of A and maximum negative value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_9	Verifying maximum positive value of A and maximum negative value of B while opcode is sub	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_10	Verifying maximum positive value of A and maximum positive value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_11	Verifying maximum positive value of A and maximum positive value of B while opcode is SUB	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_12	Verifying zero value of A and maximum negative value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_13	Verifying zero value of A and maximum negative value of B while opcode is SUB	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_14	Verifying zero value of A and maximum positive value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_15	Verifying zero value of A and maximum positive value of B while opcode is SUB	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_16	Verifying zero value of A and zero value of B while opcode is ADD	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_17	Verifying zero value of A and zero value of B while opcode is SUB	Directed during the simulation	-	A checker in the testbench to make sure the output is correct

```

module ALU_4_bit (
    input  clk,
    input  reset,
    input  [1:0] Opcode,    // The opcode
    input  signed [3:0] A,  // Input data A in 2's complement
    input  signed [3:0] B,  // Input data B in 2's complement

    output reg signed [4:0] C // ALU output in 2's complement

);

reg signed [4:0] Alu_out; // ALU output in 2's complement

localparam      Add      = 2'b00; // A + B
localparam      Sub      = 2'b01; // A - B
localparam      Not_A    = 2'b10; // ~A
localparam      ReductionOR_B = 2'b11; // |B

// Do the operation
always @* begin
    case (Opcode)
        Add:      Alu_out = A + B;
        Sub:      Alu_out = A - B;
        Not_A:    Alu_out = ~A;
        ReductionOR_B: Alu_out = |B;
        default:  Alu_out = 5'b0;
    endcase
end // always @ *

// Register output C
always @(posedge clk or posedge reset) begin
    if (reset)
        C <= 5'b0;
    else
        C <= Alu_out;
end

endmodule

```

```

module ALU_tb;

    // Declare local wire and reg
    reg clk_tb;
    reg reset_tb;
    reg [1:0] opcode_tb;
    reg signed [3:0] A_tb;
    reg signed [3:0] B_tb;
    wire signed [4:0] C_tb;

    // Instantiate the module under test
    ALU_4_bit uut(
        clk_tb,
        reset_tb,
        opcode_tb,
        A_tb,
        B_tb,
        C_tb
    );

    // define local parameter for tracking the functionality
    int count_correct, count_error;

    // Define the boundaries for our No_systems( $-2^{(n-1)}$  :  $-2^{(n-1)} - 1$ ) ==> (-8--> 7)
    localparam MAXPOS = 7;
    localparam MAXNEG = -8;
    localparam ZERO = 0;

    // define clock
    localparam T = 20;

    always begin
        clk_tb = 1'b0;
        #(T/2);
        clk_tb = 1'b1;
        #(T/2);
    end

    // define task for checking
    task checking(int expected_result);
        @(negedge clk_tb);
        if (expected_result != C_tb) begin
            count_error += 1;
            $display("A = %d, B = %d, expected = %d, Actual = %d", A_tb, B_tb, expected_result, C_tb);
        end
        else begin
            count_correct += 1;
        end
    endtask

    // define task for reset functionality
    task assert_reset;
        reset_tb = 1'b1;
        checking(0);
        reset_tb = 1'b0;
    endtask

    initial begin
        clk_tb = 1'b0;
        reset_tb = 1'b0;
        A_tb = 4'b0;
        B_tb = 4'b0;

        // default case
        A_tb = ZERO; B_tb = ZERO;
        checking(ZERO);

        #10;
        assert_reset;
        #10;
        assert_reset;
    end

```

```
////////////////////////////////////
```

```
// case1  
A_tb = ZERO; opcode_tb = 2'b01;  
B_tb = ZERO;  
checking(ZERO);  
B_tb = MAXNEG;  
checking(ZERO - MAXNEG);  
B_tb = MAXPOS;  
checking(ZERO - MAXPOS);
```

```
// case2  
A_tb = MAXNEG; opcode_tb = 2'b01;  
B_tb = ZERO;  
checking(MAXNEG);  
B_tb = MAXNEG;  
checking(MAXNEG - MAXNEG);  
B_tb = MAXPOS;  
checking(MAXNEG - MAXPOS);
```

```
// case3  
A_tb = MAXPOS; opcode_tb = 2'b01;  
B_tb = ZERO;  
checking(MAXPOS);  
B_tb = MAXNEG;  
checking(MAXPOS - MAXNEG);  
B_tb = MAXPOS;  
checking(MAXPOS - MAXPOS);
```

```
////////////////////////////////////
```

```
// case4  
A_tb = ZERO; opcode_tb = 2'b10;  
B_tb = ZERO;  
checking(~ZERO);
```

```
// case5  
A_tb = MAXNEG; opcode_tb = 2'b10;  
B_tb = ZERO;  
checking(~MAXNEG);
```

```
// case6  
A_tb = MAXPOS; opcode_tb = 2'b10;  
B_tb = ZERO;  
checking(~MAXPOS);
```

```
////////////////////////////////////
```

```
// case7
```

```
A_tb = ZERO; opcode_tb = 2'b11;
```

```
B_tb = ZERO;
```

```
checking(|ZERO);
```

```
// case8
```

```
A_tb = MAXNEG; opcode_tb = 2'b11;
```

```
B_tb = MAXNEG;
```

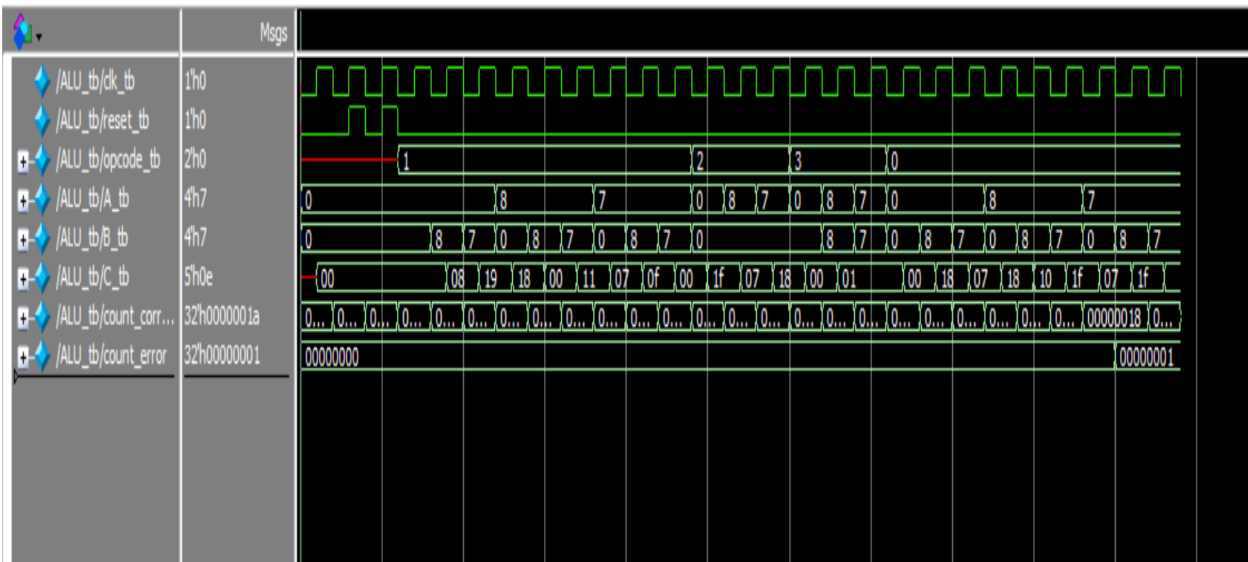
```
checking(|MAXNEG);
```

```
// case9
```

```
A_tb = MAXPOS; opcode_tb = 2'b11;
```

```
B_tb = MAXPOS;
```

```
checking(|MAXPOS);
```



```

=====
== Instance: /ALU_tb/uut
== Design Unit: work.ALU_4_bit
=====
Branch Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Branches                  7        7         0    100.00%

```

```

Statement Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Statements                 9        9         0    100.00%

```

```

Toggle Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Toggles                   44       44         0    100.00%

=====Toggle Details=====

Toggle Coverage for instance /ALU_tb/uut --

      Node      1H->0L    0L->1H  "Coverage"
      -----
      A[0-3]      1         1    100.00
  Alu_out[4-0]    1         1    100.00
      B[0-3]      1         1    100.00
      C[4-0]      1         1    100.00
  Opcode[0-1]     1         1    100.00
      clk         1         1    100.00
      reset       1         1    100.00

Total Node Count   =      22
Toggled Node Count =      22
Untoggled Node Count =      0

```

DSP_1	When the reset is asserted, the output P value should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
DSP_2	When the load is deasserted, verify the output of DSP	Randomization	-	A checker in the testbench to make sure the output is correct

```

1  module DSP(A, B, C, D, clk, rst_n, P);
2  parameter OPERATION = "ADD";
3  input  [17:0] A, B, D;
4  input  [47:0] C;
5  input  clk, rst_n;
6  output reg [47:0] P;

7
8  reg [17:0] A_reg_stg1, A_reg_stg2, B_reg, D_reg;
9  reg [18:0] adder_out_stg1, adder_out_stg2;
10 reg [36:0] mult_out;
11 reg [47:0] C_reg;

12
13 always @(posedge clk or negedge rst_n) begin
14     if (!rst_n) begin
15         // reset
16         A_reg_stg1 <= 0;
17         A_reg_stg2 <= 0;
18         B_reg <= 0;
19         D_reg <= 0;
20         C_reg <= 0;
21         adder_out_stg1 <= 0;
22         adder_out_stg2 <= 0;
23         mult_out <= 0;
24         P <= 0;
25     end
26     else begin
27         A_reg_stg1 <= A;
28         A_reg_stg2 <= A_reg_stg1;
29         B_reg <= B;
30         C_reg <= C;
31         D_reg <= D;
32         adder_out_stg2 <= adder_out_stg1;
33         if (OPERATION == "ADD") begin
34             adder_out_stg1 <= D_reg + B_reg;
35             P <= mult_out + C_reg;
36         end
37         else if (OPERATION == "SUBTRACT") begin
38             adder_out_stg1 <= D_reg - B_reg;
39             P <= mult_out - C_reg;
40         end
41         mult_out <= A_reg_stg2 * adder_out_stg2;
42     end
43 end
44
45 endmodule

```

```

1  module DSP_tb;
2
3  // declare local wire and reg
4  reg clk_tb, rst_tb;
5  reg [17:0] A_tb, B_tb, D_tb;
6  reg [47:0] C_tb;
7  wire [47:0] P_tb;
8  integer k;
9  // Instantiate the module under test
10 DSP uut(
11     A_tb,
12     B_tb,
13     C_tb,
14     D_tb,
15     clk_tb,
16     rst_tb,
17     P_tb
18 );
19
20 // General stimuli
21
22 always #5 clk_tb = ~clk_tb;
23
24 task assert_reset;
25     rst_tb = 1'b0;
26     @(negedge clk_tb);
27     if (P_tb == 48'b0) begin
28         $display("Reset successful");
29     end
30     rst_tb = 1'b1;
31 endtask
32
33 function [47:0] golden_model;
34     input [17:0] A, B, D;
35     input [47:0] C;
36     reg [17:0] add_result;
37     reg [36:0] mult_result;
38     begin
39         add_result = B + D;
40         mult_result = A * add_result;
41         golden_model = mult_result + C;
42     end
43 endfunction
44

```

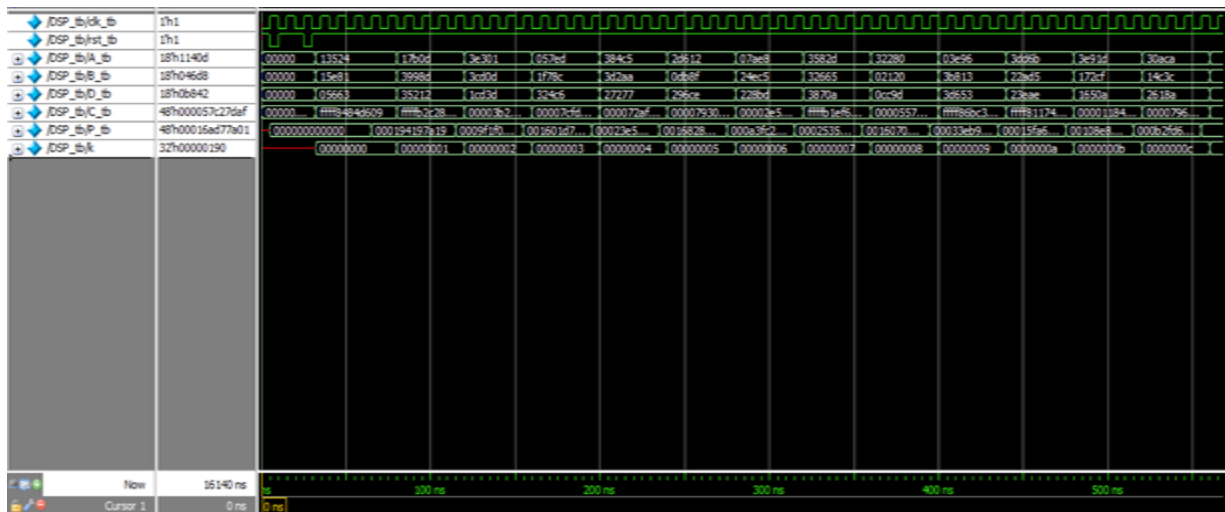
```
task check_result();
    reg [47:0] expected_result;
    expected_result = golden_model(A_tb, B_tb, D_tb, C_tb);
    repeat(4) @(negedge clk_tb);
    if (P_tb != expected_result) begin
        $display("Test failed: A = %d, B = %d, C = %d, D = %d, Expected P = %d, Actual P = %d", A_tb, B_tb, C_tb, D_tb, expected_result, P_tb);
    end else begin
        $display("Test passed: A = %d, B = %d, C = %d, D = %d, P = %d", A_tb, B_tb, C_tb, D_tb, P_tb);
    end
endtask

initial begin
    clk_tb = 1'b0;
    rst_tb = 1'b1;
    A_tb = 0;
    B_tb = 0;
    C_tb = 0;
    D_tb = 0;

    #5;
    assert_reset;
    #15;
    assert_reset;
    #2;
    for(k = 0; k < 400; k = k + 1) begin
        A_tb = $random;
        B_tb = $random;
        C_tb = $random;
        D_tb = $random;
        #10; // wait for the result to stabilize
        check_result();
    end

    #100;
    $stop;
end

endmodule
```



==== Instance: /DSP_tb/uut
==== Design Unit: work.DSP
====

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Branches	8	8	0	100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	24	24	0	100.00%